

LLM 과 효과적으로 대화하기: 프롬프트, 컨텍스트, 하네스 엔지니어링

강의 1 부: 배경, 발전의 역사, 그리고 개념 정리

저자 소개:

- 정갑주 (jeongk@konkuk.ac.kr & karpjoo@gmail.com)
- 소속:
 - 건국대학교 컴퓨터공학과
 - 인간 중심 스마트 인프라 연구소

유의사항:

1. 모든 강의자료는 **저자의 개인적 지식, 경험 및 의견**을 토대로 작성되었습니다.
- 그러나 AI 기술이 많은 기업과 전문가들에 의해서 비약적으로 발전하고 있는 상황에서 저자 혼자서 알 수 있는 것은 솔직히 매우 제한적일 수 밖에 없습니다. 따라서 더 이상 의미가 없는 과거 기술에 대한 설명, 최신 기술에 대한 잘못된 해석, 충분히 인정받지 못한 주관적 견해들이 포함되어 있을 수 있습니다.
- 최대한 객관적이고 정확하게 설명하려고 노력은 했지만, 현 시점의 AI 기술 상황에서는 한계가 있을 수 밖에 없었다는 핑계로 양해를 구합니다.
2. 모든 강의자료의 주제와 내용은 저자가 직접 결정한 것들입니다. 그러나 **포함된 참고자료, 사례조사 및 예시는 LLM 을 사용해서** 만들었습니다.
- 그런데 LLM 은 사실이 아닌 것을 그럴싸하게 지어내는 실수 (환각) 를 자주 합니다. 따라서 강의자료의 참고자료, 사례, 및 예시에 오류가 있을 수 있습니다.
- 그런 LLM 의 실수는 모든 출처를 철저히 하고 조사해야만 확인이 가능한데, 현재 강의자료를 만들고 있는 과정이라 아직 완전하게 검증하지 못했습니다. 혹시 오류가 있다면 미리 이해를 구합니다. 오류는 저자에게 알려주시길 부탁드립니다.

0. 이 강의의 위치와 목적

- **대상:** 컴퓨터공학 3-4 학년
- **위치:** Agentic Coding 커리큘럼—프롬프트/컨텍스트/하네스 엔지니어링 모듈의 **도입부 (배경 및 전제)**
- **선행 요구:** LLM 기초 (다음 토큰 예측, Pre-/Post-Training 의 존재), AI Agent 개념 개괄
- **후속 연결:** 하네스 엔지니어링 심화, A-SDLC, 인지 스택 (L0-L5)

우리는 지금 **LLM 에게 일을 시키는 방법**을 배우려고 합니다. 그런데 시작하기 전에 주목해야 할 점이 하나 있습니다.

지금까지 컴퓨터 기술 (예: 컴파일러) 사용법을 배울 때는 그 기술을 공부하면 되었지, 이렇게 **따로 방법론 과목을 공부할 필요**가 없었습니다. 기술 매뉴얼을 읽고, 옵션을 외우고, 그대로 쓰면 되었습니다. 그런데 LLM 에는 따로 “프롬프트 엔지니어링”이니 “컨텍스트 엔지니어링”이니 하는 이름이 붙은 별도의 **LLM 사용 방법에 관한 기술**이 생겼습니다. 왜 이 기술만 유독 **사용법을 따로 연구해야** 하는 기술이 되었을까요?

그리고 더 흥미로운 것은, LLM 을 잘 사용하기 위한 논의의 초점이 **프롬프트 → 컨텍스트 → 하네스**로 점차 확장되어 왔다는 점입니다. 세 용어가 서로를 완전히 대체한 것은 아닙니다. 이 강의에서는 이 흐름을 **서로 다른 실패를 해결하기 위해 새로운 관점이 추가된 과정**으로 해석합니다.

이 강의는 다음 두 질문에 답합니다.

1. **왜 LLM 은 다르게 배워야 하는가?**
2. **왜 프롬프트에서 컨텍스트와 하네스로 논의가 확장되었는가?** 그리고 그 셋은 무슨 관계인가?

두 번째 질문에 대한 이 강의의 해석을 미리 말씀드리면 이렇습니다. **세 개념은 단순히 더 정교해진 사용법의 단계가 아니라, 서로 다른 종류의 실패에 대응하기 위해 추가된 관점**입니다. 각각이 어떤 문제에서 또는 어떤 동기에서 필요했는지를 따라가는 것이 이 강의의 구성입니다.

1. 왜 LLM 은 다르게 배워야 하는가

1.1 기존 기술과의 진짜 차이

흔히 “기존 기술은 매뉴얼대로 쓰면 되지만 LLM 은 그렇지 않다”고 말합니다. 방향은 맞지만 정확하지 않습니다. 기존 기술도 매뉴얼만으로 되지 않습니다. 데이터베이스 튜닝, 네트워크 디버깅, 컴파일러 최적화 옵션—모두 매뉴얼 너머의 경험을 요구합니다.

차이는 **매뉴얼의 유무**가 아니라 다음 네 가지에 있습니다.

축	기존 기술	LLM
입력 → 출력	결정론적. 같은 입력이면 같은 출력	확률적. 같은 입력에도 다른 출력이 나올 수 있음
인터페이스	형식 언어 (API·문법). 위반하면 시스템이 오류를 탐지	자연어. 인간 언어의 다의성, 포괄성, 모호성 때문에 “위반”이라는 개념 자체가 없음
계약	문서화된 명세가 존재	명세가 없고 경향만 존재
실패의 양상	시끄럽다 (곧바로 탐지된다). 에러, 크래시, 스택 트레이스	조용하다 (당장은 작동되는 것처럼 보인다). 그럴듯한 오답이 나옴

핵심 이슈

네 번째 행을 특히 주목해야 합니다. **LLM의 실패는 조용합니다.** 그것이 LLM에서 가장 심각한 문제 중의 하나입니다.

- 실패가 조용하다는 뜻은 그것이 **문제를 안 일으킨다**가 아니고 **나중에 문제를 일으킨다**는 의미입니다.
- 그것은 **LLM이 실패했는지를 우리 인간이 LLM에서 응답을 받았을 때에는 탐지하기 매우 어렵다**는 의미입니다.

이러한 LLM의 **조용한 실패 (Silent Failure)**는 크게 두 가지로 분류될 수 있고 이 둘을 구분하는 것도 매우 힘듭니다.

- **조용한 모호성 (Silent Ambiguity):** 사용자의 요청 또는 요청 맥락에서 여러 가지 해석이 가능한데 LLM이 임의대로 하나의 해석을 선택해서 확신있게 응답한 경우입니다.
- **조용한 에러 (Silent Error):** LLM이 잘못된 응답을 한 경우이다. 대표적 예시가 LLM에서 환각이 발생해서 오답을 자신있게 응답한 경우입니다.
- 둘 다 모두 응답 시점에 발견하기 매우 어렵고, 발견된 상황에서도 이 둘을 구분하는 것도 매우 어렵습니다.

그런데 **기존 기술과의 차이점의 본질**은 사실 매우 간단합니다.

우리는 아직 **LLM을 제대로 이해하지 못 하고 있습니다.** 제대로 이해하지 못 한 기술을 우리가 만든 것입니다!

1.2 “이해할 수 없다”는 말의 정확한 뜻

“LLM은 블랙박스라 어떻게 작동하는지 알 수 없다”는 말을 자주 듣습니다. 이 표현은 절반만 맞습니다. 두 층위를 구분해야 합니다.

- **계산 및 작동 메커니즘:** 이것은 완전히 공개되어 있습니다. Transformer 구조, 어텐션 연산, 다음 토큰 확률 분포 등 이미 다 공개되어 있고 널리 알려져 있는 내용입니다.
- **메커니즘의 인과관계**는 불투명합니다. “이 모델이 왜 하필 이 응답을 했는가”라는 질문에 대한 답변은 아직 **인간이 이해하지 못 하는 미지의 영역**입니다. 즉, LLM이 어떤 응답을 했을 때 **어떤 이유로** 그 응답을 했는지를 모른다는 의미입니다.

이 문제가 연구가 현재 AI 분야에 주목 받고 있고, 실제 활발하게 연구되고 있는 주제인 **LLM 모델의 해석가능성 (interpretability)**입니다.

정확한 설명은 이렇습니다.

우리는 LLM이 **어떻게 계산하는지**는 정확히 압니다. 그러나 **왜 그렇게 답했는지**는 설명하지 못합니다.

이 구분이 중요한 이유가 있습니다.

- LLM을 **아예 원리 마저도 이해 불가능한 마법**으로 받아들이면, LLM에 관한 어떤 논의 (예: “왜 맥락이 필요한지”, “왜 확률적 선택이 문제인지”)도 불가능하게 됩니다.
- 어떤 논의도 **최소한의 근거 (Ground Truth)**가 있어야만 가능하기 때문입니다. 우리는 LLM의 작동 메커니즘을 토대로 이야기합니다.
- **LLM Probing**은 Transformer 구조를 이용해서 LLM 모델이 무엇을 학습했는지, 그것을 실제로 사용하는지를 파악하는 기술로서 활발하게 연구가 진행 중입니다.

1.3 문법은 없지만 규칙성은 있다

프롬프트에는 **문법이 없습니다.**

- 프로그래밍 언어의 문법에서와는 달리 따라서 어겨도 오류가 나지 않습니다.
- for 문의 괄호를 빠뜨리면 컴파일러가 거부하지만, 프롬프트를 “잘못”써도 LLM은 무언가를 답합니다.

그러나 **규칙성 (Patterns)** 은 있습니다.

- LLM 이 응답하는 것을 분석해 보면, 구조화하면 낮고, 예시를 주면 낮고, 역할을 지정하면 달라집니다. 이런 경향은 상당히 안정적으로 관찰됩니다.
- 프롬프트 작성은 이런 “**규칙성 (Patterns)**”을 활용하는 설계의 문제입니다. 이는 프로그래밍을 할 때처럼 “**확정되어 있는 규칙 (Rules)**”을 준수해야 문제가 아닙니다.
- 규칙을 지키는 일이 아니라, 규칙성을 활용해 원하는 결과를 얻을 확률 또는 결과의 질을 좋아지게 할 확률을 높이는 일입니다.

지금까지 매우 다양한 규칙성이 발견되었고, 이를 효과적으로 활용할 수 있는 다양한 기술 (프롬프트 엔지니어링) 이 개발되었다. 이 모든 것에서 핵심은 결국은 **LLM 에게 우리의 의도와 목적을 이해시키는 것**입니다.

1.4 그래서 학습 목적이 바뀝니다

LLM 은 매뉴얼에 따라 조작하는 기술이 아니라 **인간의 언어로 대화하고 이해시켜야 하는 기술**입니다.

	기존 기술	LLM
배워야 할 것	매뉴얼에 따라 정확하게 지시하는 방법	LLM 에게 의도와 목적을 이해시키는 방법
성공의 조건	문법을 지켰는가	LLM 이 내 뜻을 파악했는가
실패의 원인	사용법을 몰랐다	설명이 부정확 또는 부족했다

이 문장이 이 강의의 핵심이고 출발점입니다.

2. LLM 과의 대화를 어떻게 나눌 것인가

2.1 LLM 과의 대화와 협업: 사용자의 프롬프트 & LLM 의 응답

사용자와 LLM 과의 한번의 대화는 다음과 같이 구성됩니다.

- **프롬프트**: 사용자의 요청. 여기에는 자연어 문장뿐 아니라 도구 스키마 (JSON Schema), 도구 실행 결과, 코드, 첨부 파일 내용 이 모두 포함됩니다.
- **응답**: 프롬프트 (사용자의 요청) 에 대해 LLM 이 작성한 답변.

LLM 에 대해 이야기할 때 프롬프트는 두 가지 중의 하나를 의미합니다.

- **광의의 프롬프트**: 한 번의 LLM 호출에 들어가는 입력 전체를 포괄적으로 의미합니다. 사용자 자연어뿐 아니라 시스템 지시, 첨부 자료, 도구 스키마와 도구 실행 결과처럼 모델이 함께 참고하는 정보도 포함될 수 있습니다.
- **협의의 프롬프트**: 이 강의에서는 “**광의의 프롬프트**”를 기능에 따라 세 가지 유형으로 구분하고, 그중 하나를 협의의 프롬프트라고 부릅니다.
 - **대화의 의도 및 목적**: 협의의 프롬프트
 - **대화의 맥락**: 컨텍스트
 - **결과에 대한 성공 및 제약 조건**: 하네스

기존 기술과의 중요한 차이 중 하나는 **프롬프트의 핵심 부분이 인간의 언어 표현**이라는 점입니다.

- 시스템 API 를 호출하는 것이나 패러미터를 지정하는 것이 아니라 **인간이 자신이 원하는 것을 인간의 언어로 설명하거나 요청하는 것**입니다.
- 시스템 API 또는 패러미터에 대해 언급하더라도 그것은 **호출이나 설정이 아니고** 인간의 언어로 설명하는 것입니다.

2.2 프롬프트의 세 유형

세 가지 유형의 프롬프트는 **목적과 기능에 따라** 구분됩니다.

유형	명칭	답하는 질문	내용의 성격
① 목적	프롬프트	무엇을 원하는가	목적, 지시, 질문
② 맥락	컨텍스트	어떤 배경에서인가	맥락, 전제, 참조 자료

유형	명칭	답하는 질문	내용의 성격
③ 제약	하네스	무엇을 만족해야 하는가	성공 조건, 절차

- 이 세 유형 중에 다시 프롬프트가 포함되어 있어서, 셋을 합쳐서 **광의의 프롬프트**와 “① 유형”을 **협의의 프롬프트**로 구분해서 부르도록 하겠습니다.
- 따라서 **협의의 프롬프트**는 **광의의 프롬프트의 일부**입니다. 이 강의에서 그냥 “프롬프트”라고 하면 협의를 의미합니다.

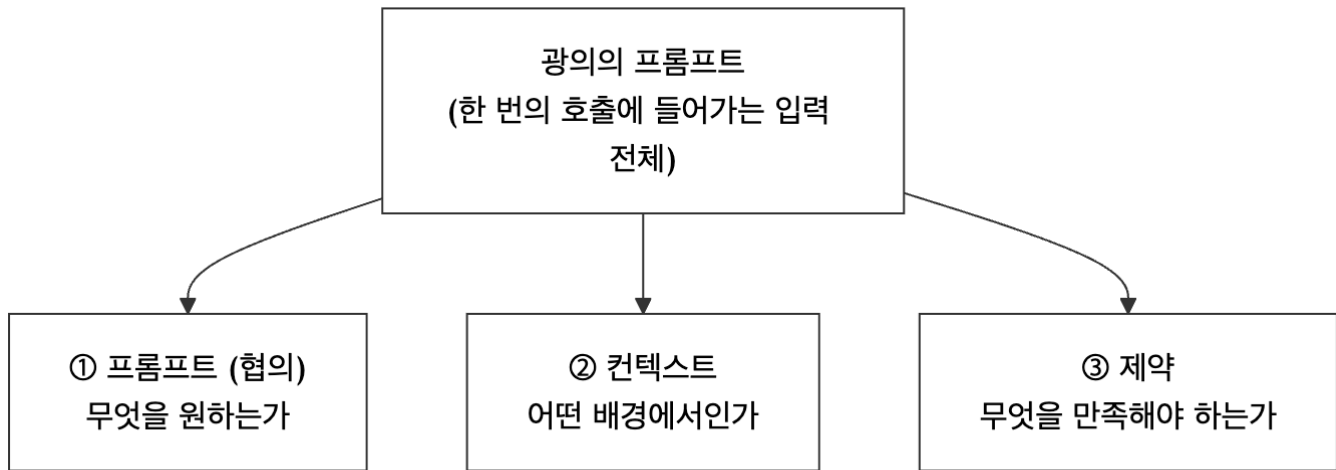


Figure 1: 프롬프트의 세 가지 유형

2.3 왜 나누는가—모델을 위해서가 아닙니다

프롬프트의 세 유형은 용어 정의나 설명으로 보면 명확하게 달라 보입니다. 그러나 실제 사용하다 보면 사실 그 차이가 명확하지 않습니다.

- **모델에게는 이 셋의 구분이 없습니다.**
- 셋 다 그냥 언어의 표현이고, 하나의 토큰 시퀀스로 들어갑니다.
- 시스템 프롬프트든 검색 결과든 도구 스키마든, 모델 내부에서 다른 종류의 것으로 취급되지 않고 인간의 언어로 표현됩니다.
- 그런데 AI Agent, 특히 **Programmed Harness**에서는 이 셋의 역할이 훨씬 명확하게 구별됩니다. 이에 대해서 Part 3에서 자세히 강의를 합니다.

그런데 이 세 유형을 구분하지 못 하면 LLM 을 제대로 사용할 수 없습니다.

- 차이점을 제대로 알 지 못하고 프롬프트를 작성하면 LLM 과의 효과적인 대화와 협력이 어렵습니다.
- 이 구분은 **모델을 위한 것이 아니라 사람의 설계를 위한 것**입니다. 모델에게는 차이가 없습니다.

3. 프롬프트 (협의) —원하는 것을 말한다

3.1 정의

프롬프트 (협의): 인간이 LLM 에게 원하는 것—의도, 목적, 수행할 지시, 답을 원하는 질문—을 **인간의 언어로 설명한 것**.

협의의 프롬프트는 가장 먼저 등장한 개념이고, 지금도 기본이고 핵심입니다.

3.2 프롬프트는 요청이지 보증이 아니다

LLM 은 매뉴얼대로 작동하는 기계가 아닙니다. 여기서 모든 것이 시작됩니다.

- **인간의 말을 듣고 자신이 이해한 것을 토대로 자신의 방식으로 응답**을 하는 시스템입니다.

- 인간이 어떤 프롬프트를 주던 최종적으로 해석하고 판단하고 결정하는 것은 LLM 입니다. LLM 이 인간의 요청을 100% 준수하는 것은 보장되지 않습니다.
- 예를 들어, “정확하게 답하라”는 지시는 정확성을 만들지 않습니다. LLM 에게 정확성을 부탁하는 것이지 지시하는 것은 아닙니다.

그동안 LLM 을 많은 사람들이 사용했고, LLM 이 인간의 요청 (프롬프트) 에 어떻게 대응하는 지에 대해서는 이제 어느 정도 파악이 되었습니다.

- 효과적으로 프롬프트를 만드는 기술을 **프롬프트 엔지니어링**이라고 합니다.

프롬프트 엔지니어링이 활용되면서 문제점과 한계가 드러났습니다.

- **맥락 필요성:** “원하는 것”을 설명할 때 **배경, 상황, 근거 같은 맥락**을 포함하고 안 하고에 따라서 LLM 응답의 수준에 영향을 크게 줄 수 있다는 사실이 확인되었습니다.
- **제약 필요성:** 그러나 LLM 모델이 최종적으로 요청받았던 대로 결과를 만들었는지를 **확인할 수는 없습니다.** 이는 LLM 의 생성형 알고리즘의 원리 때문에 어쩔 수 없습니다.

이 두 가지 사실 때문에 협의의 프롬프트에 추가해서 컨텍스트와 하네스가 제시되었습니다.

3.3 예제로 이해하기—우선 원하는 것만 말해 보기

앞으로 §4 와 §5 에서도 **같은 두 사례**를 계속 사용하겠습니다. 처음에는 의도적으로 프롬프트만 줍니다.

예제 A —코딩: 학생 성적 계산 프로그램

>> 프롬프트:
"학생들의 점수를 입력받아 최종 성적을 계산하는 Python 프로그램을 만들어 줘."

이 요청만으로도 LLM 은 무엇인가를 만들 수 있습니다. 그러나 중요한 정보가 빠져 있습니다.

- 중간고사, 기말고사, 과제, 출석의 반영 비율은 얼마인가?
- 각 항목의 점수 범위는 무엇인가?
- 잘못된 입력은 어떻게 처리할 것인가?
- 최종 점수만 필요한가, 학점 (A, B, C 등) 까지 필요한가?

즉 **무엇을 원하는가**는 어느 정도 전달했지만, **그 일을 어떤 상황과 규칙에서 해야 하는가**는 충분히 설명하지 않았습니다.

예제 B —일반: 제주도 2 박 3 일 가족여행

>> 프롬프트
"제주도 2박 3일 여행 계획을 만들어 줘."

LLM 은 유명 관광지과 식당을 조합해 그럴듯한 일정을 만들 수 있습니다. 그러나 이 요청만으로는 다음을 알 수 없습니다.

- 누구와 함께 가는가?
- 많이 걷는 여행이 가능한가?
- 렌터카를 사용하는가?
- 숙소는 어디인가?
- 관광, 음식, 휴식 중 무엇을 더 중요하게 생각하는가?

여기서 핵심은 간단합니다.

- **프롬프트는“무엇을 원하는가”를 말합니다.**
- 그러나 원하는 것만 말했다고 해서 LLM 이 **내가 머릿속에서 암묵적으로 가정한 상황까지 아는 것은 아닙니다.**

다음 §4 에서는 똑같은 두 사례에 **컨텍스트**를 추가해 보겠습니다.

4. 컨텍스트—원하는 것에 대한 맥락을 채운다

4.1 발견된 문제: 침묵하는 모호성

프롬프트만 쓰던 초기에 두 가지 현상이 관찰되었습니다.

- 인간의 언어로 명확히 썼다고 생각한 프롬프트에 **엉뚱한 답변**이 돌아왔습니다.
- 설명을 더 길게 붙였는데도 **나아지지 않거나 오히려 나빠지는** 경우가 있었습니다.

처음에는 **LLM의 성능 부족 또는 오작동**으로 생각했지만, 결국 문제의 핵심적 원인은 인간 쪽에 있는 것, 즉 **프롬프트를 어떻게 설계하느냐에 따라 결정**된다는 것으로 밝혀지고 있습니다.

- 프롬프트를 어떻게 만드느냐에 따라 동일한 LLM에서 받는 응답의 수준이 크게 달라지는 것을 보면, 이런 진단이 맞다는 사실은 쉽게 알 수 있습니다.

“프롬프트로 원하는 것을 요청할 때” 인간이 흔히 하는 실수는 **그것에 관한 맥락 (배경, 상황, 근거 등)**을 명시하지 않는 것입니다.

- 이런 실수는 인간 사이의 대화에서도 자주 있습니다. 그런 식으로 대화하는 사람들은 “너는 말을 할 때 맥락이 없냐”는 비난을 자주 받습니다.
- 예를 들어, “이 코드 좀 개선해 줘”라는 요청에는 다음과 같은 다양한 가정이 암묵적으로 전제되어 있습니다.
 - 어떤 언어의 관례를 따를지, 성능과 가독성 중 무엇이 우선인지, 기존 인터페이스를 바꿔도 되는지, 이 코드가 어디에 쓰이는지를 개발자는 생각하고 있습니다.
 - 그런데 이런 것들은 보통 프로젝트 초반에 또는 필요 시에 전체 팀 회의에서 협의되고 확정되고 공유되기 때문에, 개인적으로는 크게 신경쓰지 않고 정해진 것을 따르기만 하면 됩니다.

그런데 이런 것들을 LLM에게 명확하게 설명하지 않으면 문제가 됩니다. 이렇게 필요한 정보가 누락되어서 요청이 모호하더라도 LLM은 **되묻는다고 보장되지 않으며**, 모호한 요청 또는 부족한 정보에 대해서는 임의대로 판단하거나 선택해서 응답할 수 있기 때문입니다.

- 요청이 모호하다는 것은 **다양한 해석이 가능하고** 정보가 부족하다는 것은 그것에 대해 **다양한 선택을 할 수 있다**는 의미입니다. 그런데 이런 경우에 LLM은 확률적 선택을 해서 그럴듯한 답을 만들어 냅니다.
 - 사실 이것은 **LLM이 자율적으로 작업을 할 수 있는 능력**의 토대입니다.
- 이 현상을 앞에서 언급했던 **조용한 또는 침묵하는 모호성 (silent ambiguity)**이라고 부릅니다.
 - 이때 LLM의 확률적 선택이 요청의 의도와 맥락에 부합될 수도 있고, 아닐 수도 있습니다.
 - 동일한 요청을 여러 번 입력하면 매번 다른 응답이 나올 수 있습니다.
 - 이전 강의에서 분명히 지적했지만, 이 문제는 LLM의 문제점이거나 오류가 아니고 **Post-Training 과정에서 의도된 LLM의 설계, 즉 학습의 결과**입니다.
- 이런 문제가 코딩에서는 개발 시점에는 잘 드러나지 않고 있다가 나중에 테스트 시점, 또는 더 심각하게는 운영 시점에 발견될 수 있습니다.

이런 침묵하는 모호성을 대처하기 위해 **컨텍스트 엔지니어링** 기술이 제안되었습니다.

4.2 해결방안: 컨텍스트 엔지니어링

컨텍스트:

- 프롬프트가 성립하기 위해 전제되어야 하는 **배경, 상황 또는 근거**와 같은 맥락 정보를 의미한다.
- 인간이 **암묵적으로 가정**하고 있으나 **프롬프트에 표현하지 않았던 것**을 명시적으로 프롬프트에 포함시키는 것입니다.

관련 있는 컨텍스트를 명시적으로 제공하면 많은 과제에서 모델의 응답 품질이 좋아질 수 있다는 것은 여러 연구와 실험에서 관찰됩니다.

- 다만 효과의 크기는 모델, 과제, 컨텍스트의 종류와 길이에 따라 달라집니다. 따라서 “컨텍스트를 추가하면 항상 일정한 만큼 좋아진다”는 보편적인 법칙으로 이해하면 안 됩니다.

응답 수준이 개선되는 이유

- LLM의 **확률적 생성 과정**은 응답을 생성할 때 **다음 토큰에 관한 선택**을 반복합니다.
- 이런 “확률적 선택”을 할 때마다 **사용자가 입력한 프롬프트 내용과 그 시점까지 생성한 토큰 시퀀스**를 근거로 결정합니다.
- 따라서 **추가적인 맥락 정보**는 다음 토큰의 확률 분포에 영향을 주어, 모델의 생성 과정이 **사용자의 의도와 목적을 더 잘 반영하도록 유도**할 수 있습니다.

4.3 같은 예제에 컨텍스트를 추가하기

§3의 두 프롬프트는 틀린 요청이 아니었습니다. 다만 사용자가 알고 있는 중요한 배경을 LLM은 알지 못했습니다. 이번에는 **프롬프트는 그대로 두고, 필요한 컨텍스트를 추가**해 보겠습니다.

예제 A —코딩: 학생 성적 계산 프로그램

```
>> 프롬프트:
"학생들의 점수를 입력받아 최종 성적을 계산하는 Python 프로그램을 만들어 줘."

>> 컨텍스트:
"
이 프로그램은 대학 수업의 성적 계산에 사용한다.
중간고사 30%, 기말고사 40%, 과제 20%, 출석 10%를 반영한다.
각 항목은 0~100점이다.
우선 최종 점수만 계산하고 학점(A, B, C 등)은 계산하지 않는다.
"
```

이제 LLM 이 임의로 정해야 할 것이 크게 줄었습니다.

- 가중치를 스스로 추측할 필요가 없습니다.
- 점수 범위를 추측할 필요가 없습니다.
- 학점까지 계산해야 하는지 판단할 필요가 없습니다.

여기서 **LLM 의 Python 실력이 갑자기 좋아진 것은 아닙니다.** 사용자가 이미 알고 있던 중요한 정보를 제공했기 때문에, 모델이 사용자의 의도에 맞는 코드를 만들 가능성이 높아진 것입니다.

예제 B —일반: 제주도 2 박 3 일 가족여행

```
>> 프롬프트
"제주도 2박 3일 여행 계획을 만들어 줘."
>> 컨텍스트
"
부모님 두 분과 함께 가는 가족여행이다.
부모님은 오래 걷기 어렵다.
렌터카를 이용하고 숙소는 서귀포에 있다.
유명 관광지를 많이 방문하는 것보다 좋은 경치와 음식을 여유 있게 즐기는 것이 목적이다.
"
```

같은“제주도 2 박 3 일 여행”이지만 결과는 달라져야 합니다. 긴 등산 코스나 하루에 관광지를 여러 곳 몰아서 방문하는 일정은 이 컨텍스트와 잘 맞지 않습니다.

두 사례에서 컨텍스트의 역할은 같습니다.

	코딩 사례	일반 사례
프롬프트	무엇을 만들 것인가	어떤 여행을 계획할 것인가
컨텍스트	계산 규칙과 사용 상황	동행자·이동수단·숙소·여행 성향
효과	임의의 구현 선택을 줄임	임의의 일정 선택을 줄임

그러나 아직 한 가지 문제가 남아 있습니다.

컨텍스트를 충분히 제공했다는 것과, 최종 결과가 실제로 그 조건을 만족한다는 것은 같은 말이 아닙니다.

성적 프로그램이 정말 30%·40%·20%·10% 로 계산했는지, 여행 일정이 정말 부모님이 감당할 수 있는 이동량인지 **별도로 확인하지 않으면 알 수 없습니다.**

이 문제가 §5 의 하네스로 이어집니다.

4.4 중요한 오해: “얼마나 상세하게”가 문제가 아니라 “얼마나 관련되어 있나”가 문제다

설명을 자세히 할수록 더 틀린다는 경험담을 자주 듣습니다. 맞는 말도 틀린 말도 아닙니다.

- 이 말은 문자 그대로 받아들이면 “프롬프트는 짧게 써야 한다”는 잘못된 결론에 도달합니다.
- 그러나 이것은 잘못된 일반화입니다.

정확한 의미는 아래 상황을 고려한 조건부적 주장입니다.

- **관련 있는 정보를 추가하면** 성능은 좋아집니다. RAG와 Few-Shot이 작동하는 이유가 이것입니다.
- **무관한 정보를 추가하면** 크게 나빠집니다. Shi 등 (2023)은 초등 수학 문제 (GSM8K)에 문제 풀이와 무관한 문장을 섞은 GSM-IC 데이터셋을 만들었습니다. 원래는 맞았던 문제도 무관한 정보 한 조각 때문에 정확도가 크게 떨어졌고, Chain-of-Thought 프롬프팅을 적용해도 이 교란에서 자유롭지 않았습니다.
- **정보의 위치도 영향을 줍니다.** 긴 컨텍스트의 중간에 놓인 정보는 앞뒤에 놓인 정보보다 활용도가 떨어집니다 (Liu 등, 2023).

4.5 컨텍스트 엔지니어링

컨텍스트 엔지니어링은 “프롬프트에 맥락 정보를 추가”할 때 **필요 이상으로 길게 만들거나 과도하게 줄이는 것이 아니라, 현재 작업에 관련 있는 정보를 선별해 포함시키는 것**입니다.

오염 경고

- 컨텍스트 엔지니어링은 “대충 아무 정보나 넣어 보고 결과가 어떤지를 판단하는 기술”이 아닙니다.
- **불필요한 정보, 관련이 없는 정보**는 LLM이 사용자가 의도한 응답을 하게 만드는데 도움이 전혀 되지 않습니다.
- 이런 **의미 없는 정보도 다음 토큰을 예측하는데 사용되기** 때문에 오히려 **LLM의 응답이 사용자의 의도와 목적에서 벗어나게** 만들 수 있습니다.
- 일단 LLM 응답의 중간 결과에 반영되면, 삭제나 번복이 불가능하기 때문에 이번 회차의 응답뿐만 아니라 이후의 응답들도 부정확하게 만들 수 있습니다.

초점 맞추기

- 프롬프트에서 “원하는 것”에 **맥락 정보를 추가**할 때는 “카메라 초점을 맞추듯이” LLM이 사용자가 원하는 것을 좀더 명확하게 설명하는데 도움이 될 수 있는 정보만 포함시켜야 합니다.
- “컨텍스트 엔지니어링”은 **반드시 필요한 정보**를 포함시키고 **불필요한 정보** 걸러내는 필터링 기술입니다.

4.6 컨텍스트의 한계

프롬프트 내에 추가된 컨텍스트가 분명하게 LLM이 사용자의 의도와 목적에 더 가까운 응답을 하게 만들게 합니다. 그러나 중요한 한계가 있습니다.

- 컨텍스트는 정보의 **가용성 (availability)**을 만들지, 정보의 **사용 (use)**을 보증하지 않습니다.
- 이것은 **확률적 선택이 갖는 이론적 한계**입니다.

예를 들어, “특정 규약 문서를 컨텍스트에 넣었다는 것”은 LLM 모델이 그것을 **고려한다**는 뜻이지, **지킨다**는 뜻이 아닙니다.

- 프롬프트가 “요청이지 보증이 아니”였듯이, 컨텍스트는 “제공이지 사용이 아닙니다.”

이 한계를 대처하기 위해 세 번째 기술, **하네스 엔지니어링**이 제안됩니다.

5. 하네스—만족해야 할 조건과 따라야 할 절차

5.1 해결되지 않는 문제

프롬프트를 더 명확하게 만들고 **관련 있는 컨텍스트**를 충분히 제공하면, 일반적으로 응답이 사용자의 기대에 가까워질 가능성이 높아집니다. 그러나 이것만으로 **모든 성공 조건이 항상 만족된다고 보장할 수는 없습니다**. 한 조건을 개선했지만 다른 조건이 빠지거나, 실행할 때마다 일부 조건의 준수 여부가 달라질 수도 있습니다. LLM의 응답에서 아래 현상이 완전하게 제거되지 않습니다.

- **오류의 위험:** 대부분 맞는 응답 속에도 일부 틀린 내용이 들어갈 수 있다.
- **부정확성의 위험:** 필요한 응답 속에 불필요하거나 관련이 없는 내용이 일부 포함될 수 있다.

5.2 해결방안: 하네스 엔지니어링

컨텍스트 엔지니어링은 사용자가 원하는 것의 맥락을 제공함으로써 LLM이 응답 과정에서 수행하는 확률적 선택이 사용자의 목적을 좀더 반영하게 하는 방법입니다.

- 그러나 확률적 선택은 이론상으로 어떤 선택을 100% 강제할 수 없습니다.

그래서 하네스는 **확률적 선택에 대한 영향**이 아니라 **응답 절차와 결과가 만족해야 할 조건, 제약을 명시하는 것으로** 이 문제를 해결하려는 것입니다.

- **승인 조건 (성공 조건):** 무엇을 “통과”로 볼 것인가
- **절차 규정:** 어떤 순서와 단계를 거칠 것인가

핵심 포인트: 자율에서 통제로 - 개념적으로는 “하네스”는 **컨텍스트의 특수한 경우**라고 해석할 수 있습니다. - “성공 조건과 절차” 또한 맥락이기 때문입니다. - 그러나 하네스는 AI 에 대한 관점을 **자율 대신에 통제**로 전환하는 것입니다. - 이것이 이 강의에서 가장 중요한 요점입니다.

5.3 컨텍스트와 하네스의 차이점

(1) 컨텍스트로 하네스를 구현하기

개념적으로는 컨텍스트로써 LLM 을 통제하는 것이 어느 정도 가능합니다.

- 프롬프트에 LLM 이 지켜야 할 응답 절차 그리고 최종 결과가 만족해야 할 조건을 구체적으로 제시할 수 있습니다.
- LLM 은 실제 이런 요청을 반영하도록 훈련되어 있기 때문입니다.

(2) Pre-Training 과 Post-Training

- “Pre-Training”은 세상에 대한 지식과 언어 능력의 기반을 만듭니다.
 - 이 단계의 베이스 모델은 **지시를 안정적으로 따르도록 정렬된 모델이 아닙니다.**
 - 기본적으로 주어진 텍스트 뒤에 이어질 가능성이 높은 토큰을 생성합니다. 따라서 질문 형식의 입력 뒤에 답처럼 보이는 텍스트가 이어질 수는 있지만, 오늘날의 대화형 모델처럼 사용자의 지시를 일관되게 따르는 행동은 충분히 형성되어 있지 않습니다.
- “Post-Training(지시 튜닝, 선호도 학습·강화학습 등)”은 모델이 사용자의 지시와 대화 형식에 더 잘 맞추어 응답하도록 만드는 핵심 단계입니다.

그리고 여기서 중요한 해석이 나옵니다.

- **지시와 제약을 따르는 능력**은 Post-Training 이 가장 크게 개선한 능력입니다.
 - 앞으로도 지속적으로 개선될 것입니다.
- 따라서 **실용적 관점**에서는 컨텍스트로써 하네스를 구현할 수 있는 시점이 올 것이라고 예상할 수 있습니다.
 - 물론 AI 의 미래에 관한 논쟁적 주장입니다.

이에 관해서 Tesla 의 자율주행과 Waymo 의 자율주행을 비교해 보는 것이 흥미롭습니다.

- Tesla 자율주행은 카메라만 의지해서 구현됩니다.
 - 주행 중에 **카메라를 통해 수집되는 정보를 학습**해서 자율주행을 구현할 수 있다는 입장입니다.
 - 충분히 많은 정보가 수집되고 학습되면 AI 가 운전엔 필요한 수준의 능력을 학습할 수 있다는 입장입니다.
 - 사실 인간이 이런 식으로 눈으로 보면서 운전을 배웁니다.
- Google Waymo 는 카메라 영상 정보만으로 학습된 것은 전적으로 믿을 수 없다는 입장입니다.
 - 따라서 최종적 판단은 AI 모델 외부의 라이다 (LiDAR), 레이더 (RADAR), 초음파 센서 등의 물리적 하드웨어 정보에 근거해서 해야 한다는 입장입니다.

(3) 컨텍스트의 한계

그러나 컨텍스트로 하네스를 구현하는 것에는 두 가지 한계가 있습니다.

① 학습된 준수는 경향이 지 보장이 아닙니다

제약을 프롬프트에 아무리 강하게 써도 모델은 그것을 **부분적으로만 지키거나 아예 무시**하는 경우가 자주 있습니다.

- “반드시”, “절대”, “예외 없이”와 같은 단어를 아무리 사용해도 완전히 보장이 안됩니다. 실무자들이 반복적으로 겪는 현상입니다.
- 99.9% 의 준수율은 훌륭한 성능이지만, **0.1% 의 위반을 아무도 모른다면** 그것은 신뢰할 수 있는 시스템이 아닙니다.
- 100 번 중 1 번의 위반이 **언제 올지 알 수 없습니다.**
- 그리고 앞에서 설명했듯이 그 위반은 **조용히** 일어납니다. 출력은 여전히 그럴듯하기 때문입니다.
- AI 를 사용할 때 이런 위반은 빠르게 누적됩니다. 이전 작업에서 조용히 발생한 위반은 그 결과를 이용하는 다음 작업에 반영되기 때문입니다.

② 일부 제약은 애초에 학습의 대상이 아닙니다.

- 예를 들어, 아래 예시와 같은 제약에 대해 사전에 LLM 모델을 학습시키는 것을 이론상 불가능합니다.

- “우리 팀의 코드 컨벤션에 맞을 것”, “이 고객사의 톤에 맞을 것”, “이번 발표의 청중 수준에 맞을 것” 등
- 이런 조건은 **어떤 Post-Training 으로도 미리 내재화할 수 없습니다**. 모델이 우리 팀의 기준을 알 방법이 없기 때문입니다.
- 이것은 능력의 문제가 아니라 **정보의 문제**이며, 매번 컨텍스트로 제공하거나 **모델 밖에서 강제**해야 합니다.

따라서 프롬프트에서 하네스에 해당하는 성공 조건과 절차를 아무리 강하게 규정해도, LLM 이 이를 100% 준수한다고 보장할 수는 없습니다.

- “답변은 정확해야 한다” 또는 좀더 구체적으로 “답변은 유효한 JSON 이어야 하며, 스키마 X 를 만족해야 한다” 이라고 프롬프트에 규정해도 보장은 되지 않습니다.
- 비유적으로 어떤 제품의 품질 검사를 해야 하는데, **그 제품의 제작자에게** 검사를 맡기는 것과 같습니다.

하네스와 컨텍스트의 중요한 차이점

- 하네스는 제약이 준수되었는지를 판단할 **별도의 수단 또는 도구**가 필요합니다.
- 그러나 컨텍스트는 **프롬프트의 일부**이고 모델에게 위임하는 것이고 따라서 **100% 강제할 수 있는 수단이 없는 것**입니다.
- 따라서 하네스는 **모델 외부에** 제약을 판정할 수 있는 수단을 제공하는 것입니다.

(5) 핵심 포인트: 보증의 부재와 정보의 부재

이 두 가지 이유 때문에:

- 컨텍스트와 하네스를 구별해야 합니다.
- 제약 준수를 **모델 학습 (Post-Training) 에 전적으로 의존하지 않고**, **모델 바깥에서 별도로** 제공되는 수단 또는 도구로 검사하고 필요하면 차단해야 하는 이유입니다.

5.4 같은 예제에 하네스를 추가하기

이번에는 §3 과 §4 의 동일한 두 사례에 **성공 조건과 그것을 판정할 수단**을 추가해 보겠습니다.

예제 A —코딩: 학생 성적 계산 프로그램

앞에서 사용한 프롬프트와 컨텍스트는 다음과 같습니다.

- **프롬프트**: 학생들의 점수를 입력받아 최종 성적을 계산하는 Python 프로그램을 만들어 줘.
- **컨텍스트**: 중간 30%, 기말 40%, 과제 20%, 출석 10% 이며 각 항목은 0~100 점이다.

이제 성공 조건을 정합니다.

하네스의 성공 조건 예

- 가중치의 합은 1.0 이어야 한다.
- 모든 입력이 0~100 이면 최종 점수도 0~100 범위여야 한다.
- 예를 들어 중간 80, 기말 90, 과제 100, 출석 100 이면 결과는 90.0 이어야 한다.
- 0 보다 작거나 100 보다 큰 입력은 정상 점수로 처리하지 않아야 한다.

여기서 중요한 것은 이 조건을 **프롬프트에 적어 두는 것만으로 끝내지 않는 것**입니다.

예를 들어 별도의 테스트 코드가 다음을 실제로 검사하도록 만들 수 있습니다.

```
def test_grade():
    assert calculate_score(80, 90, 100, 100) == 90.0

def test_range():
    score = calculate_score(100, 100, 100, 100)
    assert 0 <= score <= 100
```

LLM 이 “테스트를 모두 만족했습니다”라고 말하는 것과 **테스트 프로그램을 실제로 실행해서 통과한 것**은 다릅니다.

프롬프트/컨텍스트: LLM 이 올바른 코드를 만들도록 돕는다.

하네스: 만들어진 코드가 정해진 조건을 만족하는지 실제로 검사한다.

예제 B —일반: 제주도 2 박 3 일 가족여행

앞에서 제공한 컨텍스트를 바탕으로 다음과 같은 성공 조건을 정할 수 있습니다.

하네스의 성공 조건 예

- 하루 일정은 오전 9 시 이후에 시작한다.
- 오후 8 시 이전에 서귀포 숙소로 돌아온다.
- 하루 총 운전시간은 3 시간을 넘지 않는다.
- 방문 예정 시각에 식당이나 시설이 실제로 영업 중이어야 한다.
- 한 장소에서 다음 장소로 이동하는 시간이 일정표에 반영되어 있어야 한다.

여기에서도 단순히

“반드시 오후 8 시 전에 숙소로 돌아오게 해 줘.”

라고 LLM 에게 말하는 것만으로는 **준수를 보증하지 못합니다.**

더 강한 하네스를 만들려면, 예를 들어 지도 서비스의 이동시간 정보와 영업시간 정보를 별도로 조회하고 일정표를 검사할 수 있습니다.

- 마지막 방문지 → 숙소의 예상 이동시간을 더했을 때 20:00 이전인지 검사
- 각 장소의 도착 예정 시각이 영업시간 안에 있는지 검사
- 하루 구간별 이동시간을 합산해 3 시간 이하인지 검사
- 조건을 위반하면 일정을 승인하지 않고 다시 생성하도록 함

물론 여행 일정의“즐거움”이나“부모님이 편안하게 느끼는 정도”처럼 프로그램으로 정확히 판정하기 어려운 조건도 있습니다. 이런 조건은 사람의 검토나 LLM 기반 평가가 함께 필요할 수 있습니다.

두 사례를 한눈에 비교하기

단계	코딩: 성적 계산 프로그램	일반: 제주도 가족여행
프롬프트	성적 계산 프로그램을 만들어 줘	2 박 3 일 여행을 계획해 줘
컨텍스트	평가항목·가중치·점수 범위	동행자·숙소·교통·여행 목적
성공 조건	계산식·범위·예외 처리	이동시간·영업시간·귀환시간
검사 수단	단위 테스트·입력 검증	지도/영업정보 조회·일정 검사
핵심 질문	제대로 계산했는가?	실제 실행 가능한 일정인가?

이 두 예제는 같은 구조를 보여 줍니다.

프롬프트는 목적을 말하고, 컨텍스트는 판단에 필요한 정보를 제공하며, 하네스는 결과가 성공 조건을 만족하는지를 검사하고 필요하면 통제합니다.

6 하네스 구현 방안

응답을 제공하는 LLM 모델 바깥에 하네스를 두는 방법이 두 가지가 있습니다.

- **프롬프트 하네스:** 하네스를 프롬프트로 설계하는 것
- **프로그램 하네스:** 하네스를 프로그램으로 구현하는 것

AI Agent 에서는 이 둘을 결합해서 사용합니다.

6.1 프롬프트 하네스

(1) 설계 방안

하네스 (제약) 을 프롬프트·컨텍스트와 **한꺼번에** 주면 잘 지켜지지 않는다는 것이 실무자들의 공통된 관찰입니다. 그래서 다음과 같이 나누어 쓰는 방식이 널리 사용됩니다.

- **1 단계: 응답 생성** “프롬프트”+ “컨텍스트”(+ “하네스”) 를 주고 답변을 생성시킵니다.

- **2 단계: 응답 검증** “1 단계의 답변”+ “하네스”를 함께 주고, 만족하는지 **판정**시킵니다.

이 방법에서 중요한 점은 “2 단계”가 **1 단계 모델의 바깥에** 있어야 한다는 것입니다.

- 1 단계 종료 후 완전히 LLM 세션을 종료하고 완전히 독립적인 상태에서 새롭게 시작해서 1 단계 상태가 2 단계에 영향을 주게 해서는 안됩니다.
 - 1 단계 상태가 2 단계 응답 과정에 영향을 주게 되면, 그것은 앞에서 언급했던 “제품 제작자가 제품 품질을 검사한다”는 문제가 발생하게 됩니다.
- 더 효과적인 방법은 완전히 다른 LLM 모델에게 하네스 판정을 시키는 것입니다. 예를 들어 1 단계 Claude, 2 단계 ChatGPT 로 실행시키는 것입니다.

이 구조는 여러 이름으로 독립적으로 제안되었습니다.

이름	구조
LLM-as-a-judge	별도 호출에서 기준표에 따라 출력을 채점
Critique-revise	비평 생성 → 비평에 따라 수정
Self-Refine	생성 → 자기 피드백 → 개선 반복
Reflexion	실패 신호를 언어적 반성으로 변환해 다음 시도에 반영
에이전틱 코딩의 리뷰 단계	구현 → 별도 세션 리뷰 → 게이트 판정

(2) 2 단계 방법의 작동 원리 및 방법

- Huang 등 (ICLR 2024) 은 외부 신호 없이 모델이 자기 답을 검토·수정하는 것, **내재적 자기 교정 (intrinsic self-correction)** 은 추론 과제에서 성능을 **오히려 떨어뜨린다**는 것을 보고했습니다.
 - 제품 제작자에게 제품 품질 검사를 맡겼더니 편파적으로 판정하는 것과 유사한 상황이 발생한 것입니다.
- 2 단계가 효과를 내는 것은 **두 단계로 구성되었다는 형식** 때문이 아니라, 2 단계에서 **별도의 검증**을 하기 때문입니다.
 - 그러나 “1 단계 응답 결과를 판정”하기 위한 **명확한 검증 기준 또는 도구**가 있어야 2 단계 응답 검증이 가능합니다.

조건	예	효과
외부 검증이 있다	테스트 실행 결과, 컴파일 오류, 도구 출력, 검색된 근거	가장 강함
판정 기준이 명시적이다	체크리스트, 루브릭이 미리 주어짐	강함
작업 세션만 분리되었다	별도 세션·별도 모델. 자기 답변에 앵커링 되지 않음.	중간
위 셋이 모두 없다	“다시 한번 확인해 줘”만 요청	효과 없음 또는 역효과

마지막 경우가 실무에서 가장 주의해야 할 사례입니다.

- “한 번 더 확인해 줘”는 검증처럼 보이지만 검증이 아닙니다.

6.2 프로그램 하네스

하네스 (harness) 는 마구 (馬具) 나 안전벨트를 뜻합니다.

- 안전벨트는 운전자를 더 잘 운전하게 만들지 않습니다. **사고가 났을 때 안전을 지켜줍니다.**
- 따라서 작동이 보장되는 것이 중요합니다.
- 프로그램 하네스의 목적은 LLM 의 협조 여부와 상관없이 **검사·차단·재시도 같은 통제 절차를 실제로 실행**하는 것입니다.

프로그램 하네스는 LLM 모델 외부에 검증 프로그램을 두고, LLM 의 응답이나 행동이 정의된 제약 조건을 만족하는지를 독립적으로 판단하게 합니다.

- 프로그램으로 명시된 검사와 통제 절차는 결정론적으로 수행되도록 만들 수 있습니다.

- 다만 이것이 **시스템 전체의 절대적 정확성**을 보증한다는 뜻은 아닙니다. 잘못된 성공 조건을 정의했거나 검사 프로그램 자체에 버그가 있다면 잘못된 판정을 할 수 있습니다. 즉, **구현된 규칙의 일관된 실행과 그 규칙 자체의 타당성**은 구분해야 합니다.

프로그램 하네스는 **AI Agent의 핵심 컴포넌트**입니다. 강의자료 Part 3에서 상세하게 다룹니다. 여기서는 간략하게 소개만 합니다.

AI Agent = LLM(지능적·확률적 작동) + Programmed Harness(알고리즘적·결정론적 작동)

설계 방법

프로그램 하네스 (Programmed Harness)는 단순히 코딩으로 되는 것은 아닙니다. 하네스의 **두 가지 정의**를 이해하고 있어야 합니다.

- **기능 정의:** 목적으로 정의되는 하네스
 - 성공 조건과 절차를 기능적으로 설계하는 것.
- **실행 정의:** 실천으로 정의되는 하네스
 - 기능으로 정의된 하네스를 프로그램으로 구현한 것
 - 그 기능이 **모델 바깥에서 실제로 강제되는 상태**. 모델의 협조 없이도 성립하는 실행 체계.

두 정의의 관계는 소프트웨어공학의 **명세와 구현**의 관계와 같습니다.

- 사전조건·사후조건은 먼저 문서로 쓸 수 있고, 그것이 코드로 구현되어야 비로소 위반이 차단됩니다.
- 구현되지 않은 계약도 계약이지만, **지켜지는지 아무도 확인하지 않습니다**.

6.3 프롬프트 하네스와 프로그램 하네스의 장단점 비교

AI Agent에서 프롬프트 하네스와 프로그램 하네스는 함께 결합해서 사용됩니다. 두 가지 하네스는 상호보완적 장단점을 갖고 있기 때문입니다.

- 프롬프트 하네스는 인간의 언어로 정의할 수 있기 때문에 상대적으로 범용적이며, 더 쉽게 만들고 수정할 수 있습니다.
 - 그러나 프롬프트에 정의된 제약이 100% 지켜진다는 보장을 못 합니다.
- 프로그램 하네스는 코드로 명시된 검사·차단 절차를 **일관되고 반복 가능하게 실행**할 수 있습니다.
 - 그러나 하네스의 명세나 구현 자체가 옳바른다는 것까지 자동으로 보장되지는 않습니다.
 - 또한 하네스를 프로그램으로 설계하고 구현하는 것은 상당한 작업을 요구하고, 이후 수정 및 유지보수도 훨씬 더 복잡합니다.

6.4 하네스는 승인 조건과 절차만이 아니다

지금까지는 하네스를 성공 조건과 절차로 설명했지만, 실제 하네스는 다섯 종류가 있습니다. 이에 대해서는 Part 3에서 다룹니다.

요소	역할
성공 조건 (명세)	무엇을 통과로 볼 것인가
검사기 (오라클)	그것을 실제로 판정하는 수단
절차	판정 결과에 따른 진행·차단·재시도
경계	허용되지 않은 행위를 애초에 불가능하게 함
기록	전 과정을 증거로 남김

7. 세 개념의 관계 정리

7.1 발전의 역사—세 번의 실패와 세 번의 대응

중요: 이 순서는 더 정교해지는 발전 단계가 아닙니다. 각 단계는 **앞 단계로는 풀리지 않는 다른 종류의 실패**에 대한 대응입니다. 프롬프트를 아무리 잘 써도 컨텍스트 결핍은 해결되지 않고, 컨텍스트를 아무리 잘 채워도 검증은 생기지 않습니다.

7.2 각각이 다루는 것과 다루지 못하는 것

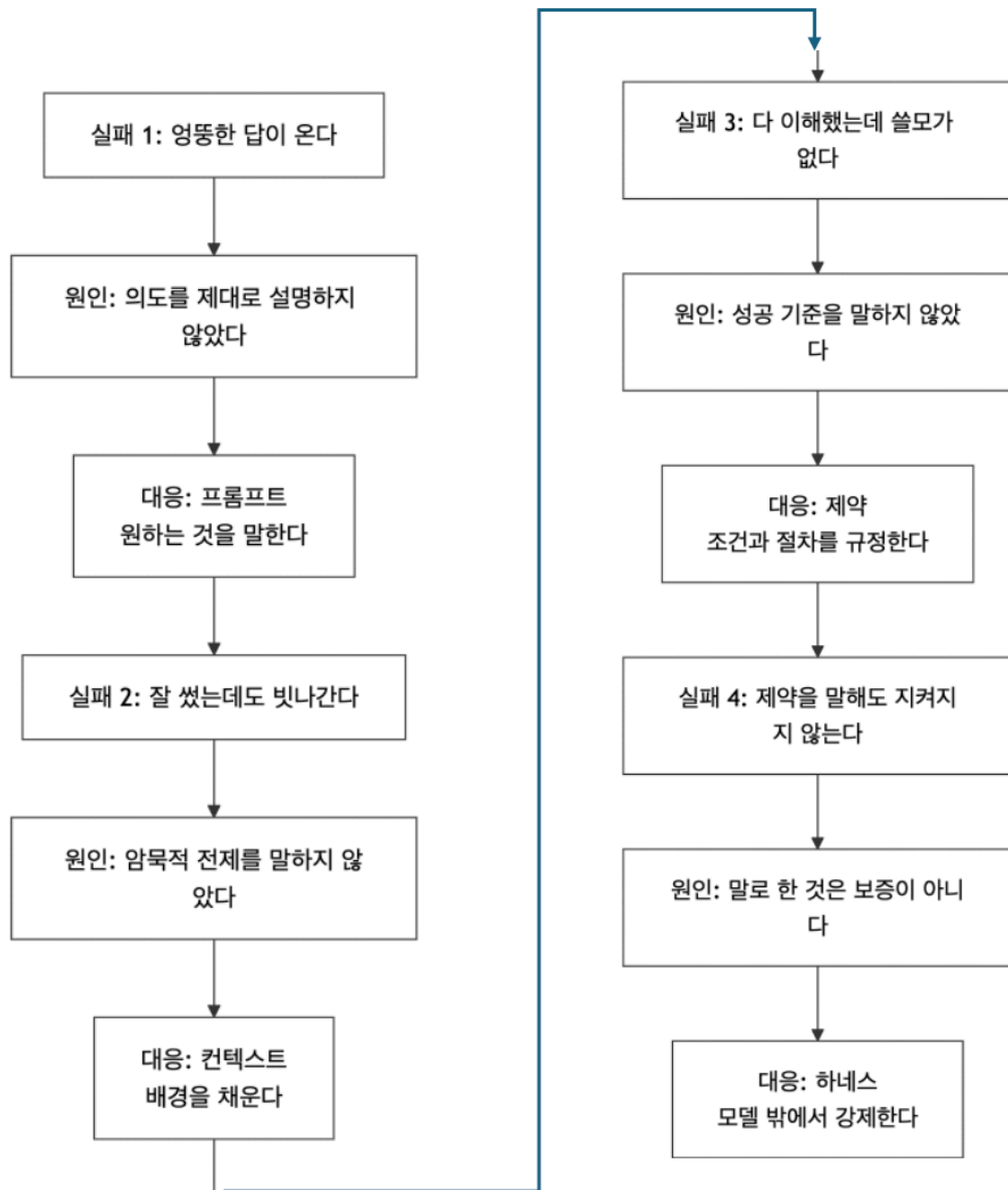


Figure 2: 프롬프트 발전 과정

	다루는 실패	한계
프롬프트	오해—요구를 잘못 이해함	모델이 모르는 것은 말해도 생기지 않음
컨텍스트	무지—필요한 사실을 모름	제공했다고 사용한 것은 아님
하네스	미검출—오류가 조용히 통과됨	게이트 자체는 자기를 검증하지 못함

- 프롬프트와 컨텍스트는 아무리 잘해도 성공 확률을 높이는 일입니다.
- 실패를 검출하는 일은 하네스 계층에서만 일어납니다.

7.3 구조적 관계

계층적 관계입니다.

- **프롬프트와 컨텍스트는 같은 계층에 있습니다.** 둘 다 토큰 시퀀스로 들어가고, 모델은 구분하지 않습니다. 그래서 둘의 경계가 흐려도 실무에 지장이 없습니다. 이 구분은 사람의 설계 편의를 위한 것입니다.
- **프로그램 하네스는 다른 계층에 있습니다.** 토큰 시퀀스 안에 들어가지 않습니다. 모델 바깥에서 실행되며, 모델의 출력을 **입력으로 받아** 판정합니다.

주의해야 할 위험입니다.

- 프롬프트와 컨텍스트의 경계는 흐려도 됩니다. **분류의 편의** 문제이기 때문입니다.
- 컨텍스트와 하네스의 경계가 흐려지면 위험합니다. **보증이 있는가 없는가**의 문제이기 때문입니다.

10. 요약

1. LLM 이 다른 이유는 매뉴얼이 없어서가 아니라 **확률적이고, 실패가 조용하기** 때문이다.
2. 우리는 LLM 이 **어떻게 계산하는지**는 알지만 **왜 그렇게 답했는지**는 모른다.
3. 프롬프트는 **요청이지 보증이 아니다.**
4. 컨텍스트는 정보의 **가용성을 만들지 사용을 보증하지 않는다.**
5. 문제는 프롬프트가 길어지는 것이 아니라 **관련 없는 내용으로** 길어지는 것이다.
6. 승인 조건은 **판정할 수단과 짝을 이룰 때만** 조건이다. 아니면 소망이다.
7. 모델은 제약을 배웠다. 그러나 **배운 것은 경향이지 보증이 아니다.**
8. 하네스에는 두 정의가 있다: **기능 정의와 실행 정의**

부록 A. 용어집

용어	정의
광의의 프롬프트	한 번의 호출에서 모델에게 주어지는 입력 전체
협의의 프롬프트	원하는 것 (의도·지시·질문) 을 인간의 언어로 설명한 것
컨텍스트	프롬프트가 성립하기 위해 전제되어야 하는 배경 정보
제약	답변이 만족해야 할 조건과 따라야 할 절차
승인 조건 (성공 조건)	무엇을 “통과”로 볼 것인가에 대한 진술
절차 제약	답변 생성 과정에서 따라야 할 순서와 단계
하네스 (기능 정의)	성공 조건과 절차를 규정하려는 기능
하네스 (실행 정의)	그 기능이 모델 바깥에서 실제로 실행이 강제될 수 있는 프로그램
Programmed Harness	AI Agent 의 결정론적·알고리즘적 성분
침묵하는 모호성	요구가 불명확한데 모델이 되묻지 않고 임의 해석으로 진행하는 현상
내재적 자기 교정	외부 피드백 없이 모델이 자기 출력을 검토·수정하는 것
검사기 (오라클)	승인 조건의 충족 여부를 실제로 판정하는 실행 가능한 수단
조용한 실패	오류가 그럴듯한 출력의 형태로 나타나 검출되지 않는 실패

부록 B. 참고 문헌

실증 근거

- Shi, F., Chen, X., Misra, K., Scales, N., Dohan, D., Chi, E. H., Schärli, N., & Zhou, D. (2023). Large Language Models Can Be Easily Distracted by Irrelevant Context. ICML 2023, PMLR 202:31210–31227. —\$4.4, \$7.8 의 근거.
- Liu, N. F. et al. (2023). Lost in the Middle: How Language Models Use Long Contexts. —\$4.4 의 위치 효과.
- Huang, J., Chen, X., Mishra, S., Zheng, H. S., Yu, A. W., Song, X., & Zhou, D. (2024). Large Language Models Cannot Self-Correct Reasoning Yet. ICLR 2024 (arXiv:2310.01798). —\$5.6 의 근거.
- Zhou, J., Lu, T., Mishra, S., Brahma, S., Basu, S., Luan, Y., Zhou, D., & Hou, L. (2023). Instruction-Following Evaluation for Large Language Models (IFEval). arXiv:2311.07911. —\$6.2 의 근거.
- Zheng, L. et al. (2023). Judging LLM-as-a-Judge with MT-Bench and Chatbot Arena. NeurIPS. —\$5.2 의 상황함 편향.
- Wei, J. et al. (2022). Chain-of-Thought Prompting Elicits Reasoning in Large Language Models. NeurIPS. —\$7.8 의 CoT ① 형.
- Ouyang, L. et al. (2022). Training Language Models to Follow Instructions with Human Feedback. NeurIPS. —\$6.2 의 근거.
- Brown, T. et al. (2020). Language Models are Few-Shot Learners. NeurIPS. —in-context learning.

2 단계 계열 (본 강의에서 원문 재확인 은 하지 않음)

- Madaan, A. et al. (2023). Self-Refine: Iterative Refinement with Self-Feedback. NeurIPS.
- Shinn, N. et al. (2023). Reflexion: Language Agents with Verbal Reinforcement Learning. NeurIPS.
- Bai, Y. et al. (2022). Constitutional AI: Harmlessness from AI Feedback.
- SCoRe (2024/2025). 강화학습 기반 자기 교정 개선 연구. Huang 등의 결론이 **영구적 한계가 아니라 현재 상태**임을 시사.

개념적 배경

- Meyer, B. (1992). Applying “Design by Contract.” IEEE Computer, 25(10), 40–51. —\$7.3 의 명세/구현 구분.
- Barr, E. T. et al. (2015). The Oracle Problem in Software Testing: A Survey. IEEE TSE, 41(5). —\$5.4 의 판정 수단 문제.
- Flavell, J. H. (1979). Metacognition and cognitive monitoring. American Psychologist, 34(10). —후속 모듈 연결.